

Trabalho 2 - DevOps

Vitória de Lira Tenório - 800381

Trabalho 2 - DevOps	1
1. Visão Geral	2
2. Tecnologias e Contêineres Utilizados	3
3. Arquitetura e Diagramas	4
Fluxo principal da aplicação	4
Arquitetura no cluster Kubernetes	5
4. Endpoints da API	6
5. Artefatos Kubernetes Utilizados	6
5.1 Secret - credenciais do banco	7
5.2 PersistentVolume e PersistentVolumeClaim - armazenamento do banco	7
5.3 ConfigMap - configuração da API	8
5.4 Deployment - quem mantém os contêineres de pé	8
5.5 Service - os nomes de rede internos	9
5.6 Ingress - a porta de entrada em k8s.local	10
6. Helm Chart	11
6.1 Estrutura de pastas do Chart	11
6.2 Valores compartilhados	11
6.3 Comando de instalação	12
7. Manual de Instalação no Kubernetes (Minikube)	12
7.1 Pré-requisitos	12
7.2 Construir e publicar as imagens	12
7.3 Subir o cluster e instalar a aplicação	12
7.4 Apontar k8s.local para o cluster	13
7.5 Observação para Windows com driver Docker	13
7.6 Acessar e verificar	13
8. Roteiro de Testes	14
8.1 Componentes de pé	14
8.2 A interface abre pelo Ingress	14
8.3 Enviar uma URL e ver o fluxo da fila	14
8.4 O worker processou e o resultado ficou em cache	14
8.5 Histórico e ranking vêm do banco	15
8.6 Testes das funcionalidades da aplicação	15
8.7 Resumo do roteiro	16

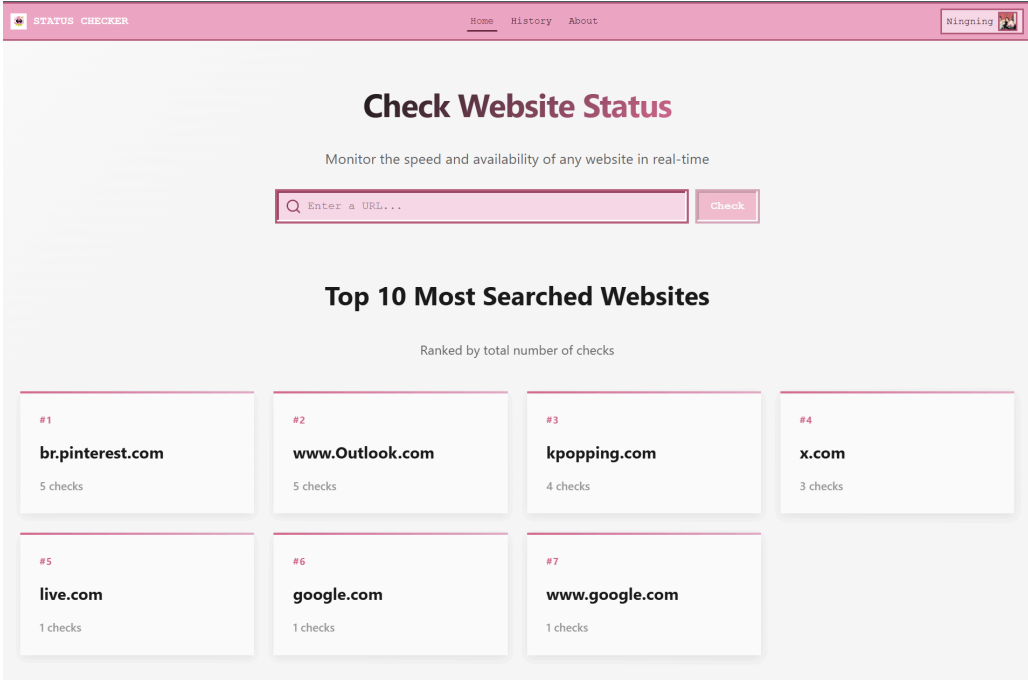
1. Visão Geral

O HeatLink é uma aplicação web para verificação de disponibilidade de URLs. O sistema recebe uma URL, verifica se há um resultado recente em cache e, caso não haja, enfileira a URL para processamento assíncrono por um worker dedicado. Os resultados são

armazenados em banco de dados e podem ser consultados posteriormente, junto com um ranking das URLs mais verificadas. O fluxo principal da aplicação é o seguinte:

1. O usuário submete uma URL pelo frontend.
2. A API verifica se existe um resultado recente no cache Redis (cache hit). Em caso afirmativo, retorna imediatamente.
3. Se não houver cache (cache miss), a URL é enfileirada no Redis e a API retorna status "pendente".
4. O worker consome a fila, realiza a verificação HTTP da URL e persiste o resultado no PostgreSQL.
5. O usuário pode consultar o histórico de resultados e o ranking das URLs mais acessadas.

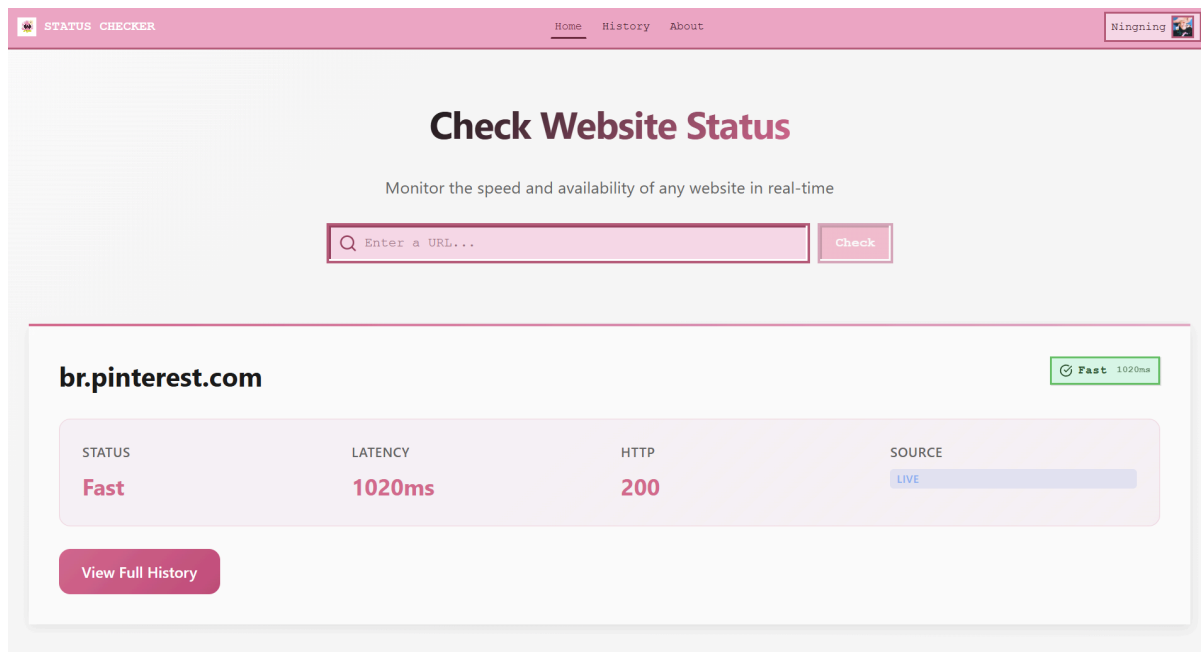
Home page



The screenshot shows the home page of a web application titled "STATUS CHECKER". The page has a pink header with navigation links: "Home", "History", and "About". A user profile "Ningning" is visible in the top right corner. The main content area features a large heading "Check Website Status" with a subtitle "Monitor the speed and availability of any website in real-time". Below this is a search bar with the placeholder text "Enter a URL..." and a "Check" button. Further down, there is a section titled "Top 10 Most Searched Websites" with the subtitle "Ranked by total number of checks". This section displays a grid of seven website cards, each showing a rank, the website name, and the number of checks performed.

Rank	Website	Checks
#1	br.pinterest.com	5 checks
#2	www.Outlook.com	5 checks
#3	kpoping.com	4 checks
#4	x.com	3 checks
#5	live.com	1 checks
#6	google.com	1 checks
#7	www.google.com	1 checks

Resultado de pesquisa



O projeto está disponível no repositório público <https://github.com/vitorialira92/devops/>.

2. Tecnologias e Contêineres Utilizados

A aplicação é composta por cinco contêineres. No Trabalho 1 eles eram orquestrados via Docker Compose; no Trabalho 2 cada um desses componentes passou a ser executado no Kubernetes, como será detalhado nas seções seguintes.

PostgreSQL 16 (Alpine)

Banco de dados relacional responsável pela persistência dos resultados de verificação. Armazena o histórico completo de cada URL verificada e os dados utilizados para geração do ranking. É o único componente que precisa preservar dados entre reinicializações. No Docker Compose isso era feito por um volume nomeado; no Kubernetes, essa persistência passou a ser garantida por um PersistentVolume associado a um PersistentVolumeClaim.

Redis 7 (Alpine)

Atua em duas funções simultâneas: cache de resultados recentes e fila de mensagens. O cache evita requisições redundantes para URLs verificadas recentemente, com TTL configurável e padrão de 30 segundos. A fila, chamada url-queue, é implementada como uma lista Redis, na qual a API insere URLs com LPUSH e o worker as consome. Por serem dados temporários, o Redis não precisa de armazenamento persistente.

API - Spring Boot (perfil api)

Serviço backend responsável por expor os endpoints REST. Implementa a lógica de cache hit e miss, o enfileiramento de URLs e a consulta de resultados. Exposta na porta 8080. Desenvolvida em Java com Spring Boot, usando as bibliotecas Lombok, Jakarta Validation e Spring Data Redis.

Worker - Spring Boot (perfil worker)

Instância separada do mesmo artefato backend, executada com o perfil Spring worker. É responsável por consumir a fila Redis, realizar as requisições HTTP às URLs enfileiradas, registrar o resultado (status HTTP, latência e disponibilidade) e armazená-lo no PostgreSQL. Não expõe portas externas. Como a API e o worker são o mesmo programa, apenas com perfis diferentes, eles usam a mesma imagem Docker, e o que decide o papel de cada um é a variável de ambiente `SPRING_PROFILES_ACTIVE`.

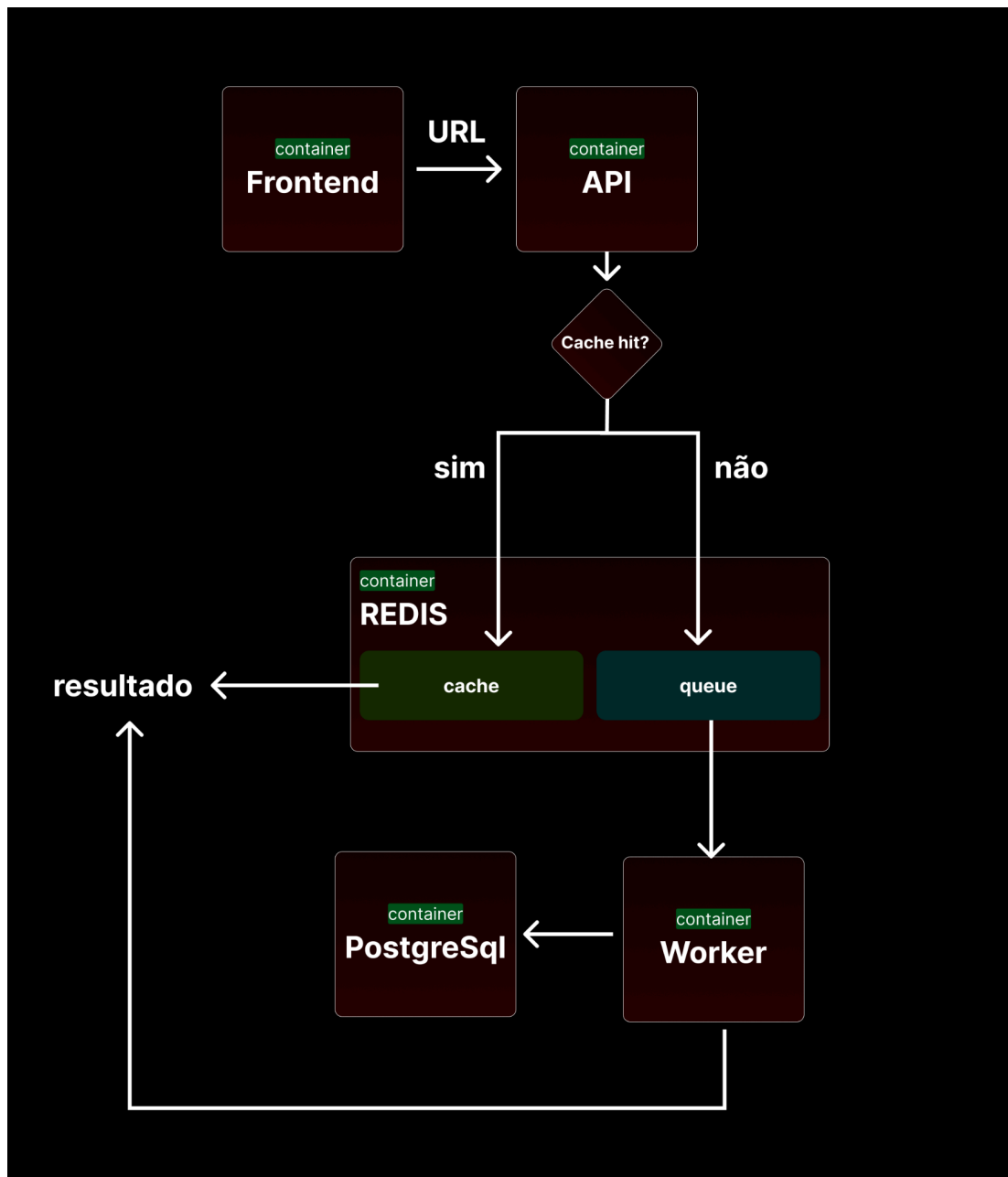
Frontend

Interface web construída em React e servida via Nginx. Permite ao usuário submeter URLs, acompanhar resultados e visualizar o ranking. Além de entregar as telas, o Nginx encaminha as chamadas dirigidas a /api para o serviço da API. No Docker Compose o frontend era exposto em uma porta local; no Kubernetes ele passou a ser publicado através do Ingress, no endereço k8s.local

3. Arquitetura e Diagramas

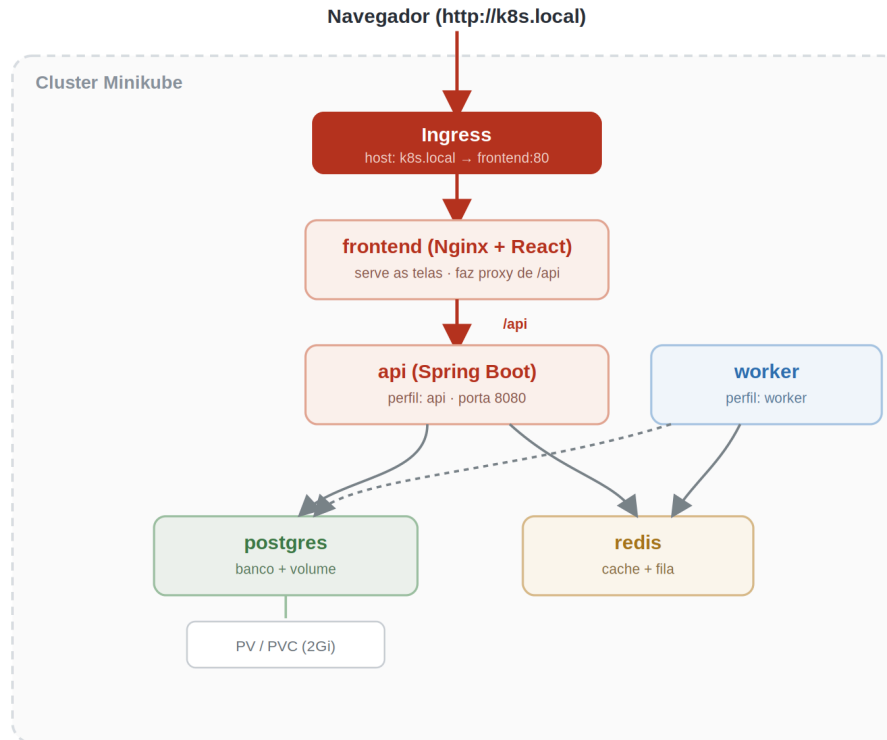
Fluxo principal da aplicação

Quando o usuário envia uma URL, a API primeiro verifica o cache no Redis. Se houver um resultado recente (cache hit), ele é devolvido de imediato. Se não houver (cache miss), a URL é colocada na fila do Redis e o worker a consome, testa o endereço, grava o resultado no PostgreSQL e o disponibiliza para consulta.



Arquitetura no cluster Kubernetes

No Kubernetes, o tráfego externo entra pelo Ingress, que o encaminha ao frontend. O frontend serve as telas e repassa as chamadas de /api para a API. Tanto a API quanto o worker se comunicam com o Redis e com o PostgreSQL, que por sua vez guarda seus dados em um volume persistente.



4. Endpoints da API

Todos os endpoints estão sob o prefixo /api.

- **POST /api/check?url={url}**: Submete uma URL para verificação. Retorna o resultado imediatamente se houver cache válido (source: cache), ou enfileira para processamento e retorna status 202 Accepted (source: queued). URLs sem esquema recebem https:// automaticamente (apesar disso não acontecer quando submetida pelo frontend devido à normalização no frontend);
- **GET /api/results/url?url={url}**: Retorna o histórico completo de verificações de uma URL específica;
- **GET /api/ranking**: Retorna as 10 URLs mais consultadas no sistema.

5. Artefatos Kubernetes Utilizados

Kubernetes trabalha com objetos, cada um com uma função específica. Esta seção descreve cada tipo de objeto criado para a aplicação e mostra o manifesto correspondente. Os mesmos objetos são gerados automaticamente pelo Helm Chart descrito na Seção 6. Ao todo, a implantação cria um Secret com as credenciais do banco, um PersistentVolume e um PersistentVolumeClaim para o armazenamento, um ConfigMap com a configuração da API, cinco Deployments (um por componente), quatro Services (todos menos o worker) e um Ingress. A tabela abaixo resume o inventário.

Tipo de objeto	Qtd.	A quais componentes pertence
Secret	1	postgres (credenciais reutilizadas por api e worker)

Tipo de objeto	Qtd.	A quais componentes pertence
PersistentVolume	1	postgres
PersistentVolumeClaim	1	postgres
ConfigMap	1	api
Deployment	5	postgres, redis, api, worker, frontend
Service	4	postgres, redis, api, frontend
Ingress	1	frontend (ponto de entrada da aplicação)

5.1 Secret - credenciais do banco

O Secret concentra o usuário, a senha e o nome do banco de dados em um único objeto, codificados em base64. Tanto o PostgreSQL quanto a API e o worker leem suas credenciais deste mesmo Secret, o que evita repetição. Vale destacar que base64 não é criptografia, é apenas um formato de armazenamento; o Secret é a forma correta de organizar credenciais no Kubernetes e de controlar o acesso a elas. O valor aGVhdGxpbms= é a palavra heatlink codificada.

None

```

apiVersion: v1
kind: Secret
metadata:
  name: heatlink-postgres-secret
type: Opaque
data:
  POSTGRES_DB: aGVhdGxpbms=
  POSTGRES_USER: aGVhdGxpbms=
  POSTGRES_PASSWORD: aGVhdGxpbms=

```

5.2 PersistentVolume e PersistentVolumeClaim - armazenamento do banco

Contêineres são descartáveis: se o pod do banco reiniciar, tudo que estava dentro dele se perde. Por isso o PostgreSQL guarda seus arquivos em um volume externo ao contêiner. O PersistentVolume representa o espaço de disco reservado, e o PersistentVolumeClaim é o pedido que o pod faz por esse espaço. O Kubernetes casa o pedido com o volume e o monta dentro do contêiner do banco.

None

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: heatlink-postgres-pv-volume
spec:

```

```

storageClassName: manual
capacity:
  storage: 2Gi
accessModes: [ReadWriteOnce]
hostPath:
  path: "/mnt/data/heatlink"
  type: DirectoryOrCreate
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: heatlink-postgres-pv-claim
spec:
  storageClassName: manual
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 2Gi

```

5.3 ConfigMap - configuração da API

O ConfigMap guarda as configurações que não são segredo: o perfil ativo e os endereços do banco e do Redis dentro do cluster. Mantê-las fora da imagem permite alterar um endereço sem reconstruir o contêiner. O usuário e a senha do banco não estão aqui, pois vêm do Secret.

```

None
kind: ConfigMap
apiVersion: v1
metadata:
  name: heatlink-api-configmap
data:
  SPRING_PROFILES_ACTIVE: "api"
  DB_HOST: "heatlink-postgres"
  DB_NAME: "heatlink"
  REDIS_HOST: "heatlink-redis"

```

5.4 Deployment - quem mantém os contêineres de pé

O Deployment é o objeto central. Ele define qual imagem rodar, quantas cópias manter e quais variáveis de ambiente injetar, e recria o pod automaticamente caso ele caia. Cada um dos cinco componentes tem o seu. Abaixo está o Deployment da API, que é o mais completo, pois combina variáveis vindas do ConfigMap e do Secret ao mesmo tempo. O trecho está condensado para caber na documentação.

None

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 1
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      containers:
        - name: api
          image: vitoriatenorio/heatlink-api:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_PROFILES_ACTIVE
              valueFrom:
                configMapKeyRef: { name: heatlink-api-configmap, key:
SPRING_PROFILES_ACTIVE }
            - name: DB_HOST
              valueFrom:
                configMapKeyRef: { name: heatlink-api-configmap, key: DB_HOST }
            - name: REDIS_HOST
              valueFrom:
                configMapKeyRef: { name: heatlink-api-configmap, key: REDIS_HOST }
            - name: DB_USER
              valueFrom:
                secretKeyRef: { name: heatlink-postgres-secret, key: POSTGRES_USER }
            - name: DB_PASS
              valueFrom:
                secretKeyRef: { name: heatlink-postgres-secret, key:
POSTGRES_PASSWORD }
```

Observação: o Deployment do worker é quase idêntico, mas define `SPRING_PROFILES_ACTIVE` como `worker` e não possui um Service associado, pois o worker nunca recebe conexões, apenas puxa tarefas da fila do Redis.

5.5 Service - os nomes de rede internos

Dentro do cluster, os pods podem ser recriados a qualquer momento e mudam de endereço quando isso acontece. O Service resolve esse problema dando um nome fixo e estável a um conjunto de pods. Existem quatro Services na aplicação, um para cada componente que precisa ser encontrado por outro; o worker é a exceção. O Service da API

precisa se chamar exatamente api, pois é assim que o Nginx do frontend o localiza ao encaminhar as chamadas de /api.

Service	Porta	Quem usa esse nome
heatlink-postgres	5432	api e worker, para acessar o banco
heatlink-redis	6379	api e worker, para cache e fila
api	8080	o Nginx do frontend e o Ingress
frontend	80	o Ingress, para publicar a aplicação

None

```
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
  selector:
    app: api
```

5.6 Ingress - a porta de entrada em k8s.local

O Ingress é o único ponto por onde o tráfego externo entra na aplicação. Ele recebe todas as requisições dirigidas ao endereço k8s.local e as encaminha para o Service do frontend. A partir daí, o próprio Nginx decide o que entregar, servindo as telas ou repassando as chamadas de /api para a API. Assim o usuário nunca precisa saber em qual porta cada componente está: tudo é acessado por uma URL única.

None

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: gateway-ingress
spec:
  rules:
    - host: k8s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend
```

```
port:  
  number: 80
```

6. Helm Chart

Aplicar sete tipos de objeto na mão, na ordem certa, é trabalhoso e fácil de errar. O Helm é o gerenciador de pacotes do Kubernetes: ele empacota todos os manifestos em um Chart, que é instalado com um único comando. O Chart deste projeto foi organizado no formato de guarda-chuva: existe um Chart principal, o heatlink, que não descreve objetos sozinho, mas reúne cinco subcharts, um para cada componente. Assim cada parte fica isolada e fácil de entender, e mesmo assim tudo sobe junto.

6.1 Estrutura de pastas do Chart

```
None  
heatlink-chart/  
  Chart.yaml      # declara o chart e seus 5 subcharts  
  values.yaml     # nomes compartilhados por todos os componentes  
  templates/  
    ingress.yaml  # o Ingress, comum a toda a aplicacao  
  charts/  
    postgres/     # secret, pv, pvc, deployment, service  
    redis/        # deployment, service  
    api/          # configmap, deployment, service  
    worker/       # deployment (sem service)  
    frontend/     # deployment, service
```

6.2 Valores compartilhados

O arquivo values.yaml do Chart principal guarda, em uma seção chamada global, os nomes que todos os componentes precisam conhecer uns dos outros. Como esses nomes ficam em um único lugar, o Deployment da API sabe o nome do serviço do banco, o Ingress sabe o nome do frontend, e assim por diante. Dentro dos modelos, esses valores aparecem entre chaves duplas e o Helm os substitui na instalação.

```
None  
global:  
  postgres:  
    name: heatlink-postgres  
    database: heatlink  
  redis:  
    name: heatlink-redis  
  api:  
    name: api      # precisa ser "api" por causa do proxy do Nginx
```

```
worker:
  name: worker
frontend:
  name: frontend
```

6.3 Comando de instalação

Com o Chart pronto, instalar a aplicação inteira, os cinco componentes e todos os objetos, se resume a um comando. Para remover tudo, usa-se o `uninstall`.

```
None
helm install heatlink heatlink-chart
helm uninstall heatlink
```

7. Manual de Instalação no Kubernetes (Minikube)

Este é o roteiro prático, do zero até a aplicação no ar em `k8s.local`. O repositório inclui scripts em `bash` que automatizam os passos abaixo.

7.1 Pré-requisitos

- Docker
- Minikube
- `kubectl`
- Helm 3

7.2 Construir e publicar as imagens

O backend e o frontend são código próprio, então é preciso transformá-los em imagens Docker. O script `build-images.sh` faz o build das duas imagens e, com a opção `-p`, também as envia para o Docker Hub. É necessário estar autenticado com `docker login` antes de usar o `-p`.

```
None
docker login
./build-images.sh -p
# constroi vitoriatenorio/heatlink-api e vitoriatenorio/heatlink-frontend
# e envia ambas para o Docker Hub
```

7.3 Subir o cluster e instalar a aplicação

Na primeira vez, o script `helm-up` é chamado com as três opções juntas: `-i` inicia o Minikube e habilita o Ingress, `-b` constrói e carrega a imagem do backend no cluster e `-f` faz o mesmo com o frontend. Ao final, ele executa o `helm install`. Carregar as imagens com

minikube image load é o que permite ao cluster usar as imagens locais sem baixá-las de novo. Nas próximas vezes, se nada mudou, basta o helm-up sem opções.

None

```
./helm-up -i -b -f
```

7.4 Apontar k8s.local para o cluster

O endereço k8s.local é um nome local que precisa ser mapeado para o IP do Minikube. Primeiro descubra o IP, depois adicione a linha ao arquivo hosts. Em Linux e macOS o arquivo é /etc/hosts; em Windows é C:\Windows\System32\drivers\etc\hosts, editado como administrador.

None

```
minikube ip
```

```
# ex.: 192.168.49.2
```

```
# adicione ao arquivo hosts:
```

```
192.168.49.2 k8s.local
```

7.5 Observação para Windows com driver Docker

Quando o Minikube roda no Windows com o driver do Docker, o IP interno do cluster não é acessível diretamente pelo navegador. Nesse caso, mantém-se um túnel aberto em um terminal à parte e o endereço no arquivo hosts passa a ser 127.0.0.1. O comando abaixo expõe o controlador do Ingress na porta 80 da máquina, e por isso o acesso continua passando pelo Ingress e pela URL k8s.local.

None

```
# terminal separado, mantido aberto (como administrador)
```

```
kubectl port-forward -n ingress-nginx service/ingress-nginx-controller 80:80
```

```
# no arquivo hosts, use:
```

```
127.0.0.1 k8s.local
```

7.6 Acessar e verificar

Com tudo no ar e o nome mapeado, abra o navegador em <http://k8s.local>. Antes de testar, confira se os cinco pods estão de pé.

None

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
api-xxxxxxxx-xxxxx	1/1	Running	0	2m

```
frontend-xxxxxxxx-xxxxx      1/1   Running 0      2m
heatlink-postgres-xxxxxxxx-xxxxx 1/1   Running 0      2m
heatlink-redis-xxxxxxxx-xxxxx  1/1   Running 0      2m
worker-xxxxxxxx-xxxxx        1/1   Running 0      2m
```

8. Roteiro de Testes

O objetivo deste roteiro é comprovar que a aplicação funciona de ponta a ponta no cluster: que a interface abre, que a API responde, que a fila e o worker processam as URLs e que os dados chegam ao banco. Os testes vão do mais simples ao mais completo, e podem ser feitos pela interface ou pela linha de comando com curl.

8.1 Componentes de pé

Rodar **kubect**l get pods e confirmar as cinco linhas com status Running e 1/1. Em seguida, **kubect**l get svc deve listar quatro Services (api, frontend, heatlink-postgres e heatlink-redis) e **kubect**l get ingress deve mostrar o gateway-ingress com o host k8s.local. Isso comprova que o Helm criou todos os objetos e que o Kubernetes conseguiu iniciar cada componente.

8.2 A interface abre pelo Ingress

Abrir <http://k8s.local> no navegador. A tela inicial do HeatLink deve carregar, com o campo para digitar uma URL. Isso comprova que o Ingress está roteando para o frontend e que o Nginx está servindo a aplicação.

8.3 Enviar uma URL e ver o fluxo da fila

Este é o teste central, pois exercita todos os componentes. Ao enviar uma URL nova, a API não devolve o resultado na hora: ela responde que a URL foi enfileirada, o que comprova que o mecanismo de fila está ativo e que a conexão entre a API e o Redis funciona.

None

```
curl -X POST "http://k8s.local/api/check?url=https://www.google.com"
```

resposta esperada na primeira vez (URL ainda nao verificada):

```
{"source":"queued",
"message":"URL enfileirada para processamento. Consulte novamente em instantes."}
```

8.4 O worker processou e o resultado ficou em cache

Poucos segundos depois, o worker já deve ter retirado a URL da fila, testado o endereço e gravado o resultado. Uma segunda consulta à mesma URL agora responde na

hora, a partir do cache. Isso comprova que o worker consumiu a fila, testou a URL e gravou o resultado, validando a cadeia entre worker, Redis e PostgreSQL.

None

```
curl -X POST "http://k8s.local/api/check?url=https://www.google.com"
```

```
# resposta esperada agora (resultado ja disponivel no cache):
```

```
{"source":"cache",  
 "result":{"url":"https://www.google.com","status":"UP", ...}}
```

8.5 Histórico e ranking vêm do banco

Por fim, as consultas de histórico e de ranking leem dados diretamente do PostgreSQL. Se elas retornam a URL testada, isso confirma que o worker persistiu os dados no banco. Como o banco usa um volume, esses dados sobrevivem mesmo que o pod seja reiniciado.

None

```
curl "http://k8s.local/api/results?url=https://www.google.com"
```

```
curl "http://k8s.local/api/ranking"
```

```
# ambos devem retornar uma lista em JSON contendo a URL testada
```

8.6 Testes das funcionalidades da aplicação

Enquanto os testes anteriores comprovam que a aplicação está no ar e que os componentes do cluster conversam entre si, esta seção trata do que a aplicação faz do ponto de vista de quem a utiliza, independentemente de estar rodando no Kubernetes ou no Docker Compose. Todos os cenários podem ser executados pela interface, acessando <http://k8s.local>, e a maioria também pela API, com os endpoints descritos na Seção 4.

Verificação de uma URL acessível: Enviar um endereço válido e no ar, como <https://www.google.com>, pelo campo da tela inicial. Após alguns segundos, o resultado deve mostrar o site como disponível, junto com o código de status HTTP retornado e o tempo de resposta. Isso comprova o caminho completo de verificação, da submissão até a exibição do resultado.

Verificação de uma URL fora do ar: Enviar um endereço que não responde, como <https://endereco-inexistente-000111.com>. O resultado deve registrar o site como indisponível, sem que a aplicação apresente erro. Isso comprova que a verificação trata corretamente tanto os acessos bem-sucedidos quanto as falhas.,

Normalização de endereço: Digitar uma URL sem o esquema, apenas google.com, e confirmar que a aplicação a aceita e a trata como <https://google.com>. Esse comportamento também pode ser observado diretamente na API, que completa o esquema automaticamente quando ele não é informado.

Comportamento do cache: Consultar a mesma URL duas vezes em sequência. A primeira consulta processa a URL e a segunda, feita logo em seguida, deve responder de forma imediata, vinda do cache. Passado o tempo de expiração, de trinta segundos por padrão, uma nova consulta volta a reprocessar a URL. Isso comprova que o cache está ativo e respeitando o tempo de vida configurado.

Ranking das mais consultadas: Verificar algumas URLs diferentes, repetindo uma delas mais vezes que as outras, e então abrir o ranking. A URL mais verificada deve aparecer no topo do Top 10, e a ordem deve refletir o número de verificações de cada endereço. Isso comprova a contagem e a ordenação do ranking.

Histórico de uma URL: Após verificar uma URL, consultar o seu histórico e confirmar que aparecem as verificações anteriores, com data, status e disponibilidade. Isso comprova que cada verificação é registrada e pode ser recuperada depois.

Navegação pela interface: Percorrer as páginas da aplicação, Home, History e About, confirmando que todas abrem e exibem os dados esperados, sem quebras de layout. Isso comprova que o frontend está servindo corretamente todas as telas.

Caso prefira testar pela linha de comando, alguns desses cenários podem ser reproduzidos com o curl:

None

URL fora do ar (deve registrar como indisponível)

```
curl -X POST "http://k8s.local/api/check?url=https://endereco-inexistente-000111.com"
```

endereco sem esquema (deve ser aceito e tratado como https)

```
curl -X POST "http://k8s.local/api/check?url=google.com"
```

ranking apos varias verificacoes

```
curl "http://k8s.local/api/ranking"
```

8.7 Resumo do roteiro

Teste	Componentes exercitados	Valida
8.1 Pods de pé	Todos	Instalação do Helm
8.2 Interface abre	Ingress, frontend	Publicação em k8s.local
8.3 Enviar URL	frontend, api, redis	Recepção e enfileiramento
8.4 Segunda consulta	worker, redis, postgres	Processamento e cache
8.5 Histórico e ranking	api, postgres	Persistência no banco

